Código Optimización (Método de Instrucciones dependientes susceptibles de reorganización).

```
import re import tkinter as tk

from typing import List, Tuple

try:

   from .lexer import lex, Token except

ImportError:

   from lexer import lex, Token



OP_PREC = {

   '*': 30, '/': 30, '%': 30,

   '+': 20, '-': 20,

}


DECLARATION_PATTERN = re.compile(r'^\s*(#Alp|@Vic|~Adv)\s+.*?;?\s*$', re.MULTILINE |
re.IGNORECASE)



class OptimizedCodeDisplay(tk.Text):


   def __init__(self, master=None, **kwargs):

      super().__init__(master, **kwargs)        self.config(state=tk.DISABLED, background="#e8f5e9",
relief=tk.FLAT, font=("Consolas", 10))        self.tag_configure("substitution_source",
background="#a8d6b1", foreground="#000000", font=("Consolas", 10, "bold"))
self.tag_configure("substitution_target", background="#69b876", foreground="#000000",
font=("Consolas", 10, "bold"))

      self.tag_configure("insertion", background="#c6e7c9", foreground="#000000", font=("Consolas", 10,
"bold"))
```

```
   def set_code(self, code: str, substitutions: List[Tuple[str, str, str, str]]):
      self.config(state=tk.NORMAL)
self.delete("1.0", tk.END)        self.insert(tk.END,
code)

      for original_expr, target_line_var, replacement_var, tag_type in substitutions:
         search_term = replacement_var.strip()

         if tag_type == "CSE_SUBSTITUTION":
            start_pos_source = self.search(search_term, "1.0", stopindex=tk.END, regexp=False)
if start_pos_source:
               end_pos_source = f"{start_pos_source}+{len(search_term)}c"
self.tag_add("substitution_source", start_pos_source, end_pos_source)

            target_line_prefix_regex = r"^\s*" + re.escape(target_line_var) + r"\s*="
start_line = self.search(target_line_prefix_regex, "1.0", stopindex=tk.END, regexp=True,
nocase=True)

            if start_line:
               search_regex = re.escape(original_expr)                start_pos_target =
self.search(search_regex, start_line, stopindex=f"{start_line} lineend", regexp=True)                if
start_pos_target:
                  end_pos_target = f"{start_pos_target}+{len(original_expr)}c"
self.tag_add("substitution_target", start_pos_target, end_pos_target)

      self.config(state=tk.DISABLED)

def get_precedence(operator: str) -> int:
   """Retorna la precedencia de un operador aritmético."""
return OP_PREC.get(operator, 0)
```

```
def simplify_rhs(rhs: str) -> str:     temp =
rhs.strip().rstrip(';')     while temp.startswith('(') and
temp.endswith(')'):
       level = 0
is_fully_enclosed = True
try:
           inner_content = temp[1:-1]
except:             break        for char
in inner_content:            if char ==
'(':               level += 1            elif
char == ')':               level -= 1
if level < 0:
              is_fully_enclosed = False
break        if is_fully_enclosed and level
== 0:             temp =
inner_content.strip()        else:
break
   return temp


def get_principal_op(expression: str) -> str | None:     current_min_prec = float('inf')     principal_op =
None     paren_level = 0


   tokens = [t.lexeme for t in lex(expression) if t.kind not in ("SKIP", "NEWLINE", "SEMI")]


   for token in tokens:
if token == '(':
```

```
        paren_level += 1        elif token == ')':
paren_level -= 1        elif paren_level == 0 and token in
OP_PREC.keys():
        prec = get_precedence(token)
if prec <= current_min_prec:
current_min_prec = prec
principal_op = token

    return principal_op

def is_safe_to_substitute(rhs: str, expr_source: str) -> bool:

    clean_rhs = re.sub(r'\s+', ' ', rhs).strip()
clean_expr = re.sub(r'\s+', ' ', expr_source).strip()

    if clean_rhs == clean_expr:
        return True

    if clean_expr not in clean_rhs:
        return False

    start_index = clean_rhs.find(clean_expr)     end_index
= start_index + len(clean_expr)

    main_op_expr = get_principal_op(expr_source)     if main_op_expr is None and
any(op in expr_source for op in OP_PREC.keys()):

        main_op_prec = 40     elif
main_op_expr is None:
```

```
        return False

    main_op_prec = get_precedence(main_op_expr)

    if start_index > 0:
        pre_expr = clean_rhs[:start_index].rstrip()
if pre_expr:
            pre_op_match = re.search(r'([+\-*/%])\s*$', pre_expr)
if pre_op_match:
                prev_op = pre_op_match.group(1)
prev_prec = get_precedence(prev_op)

                if prev_prec > main_op_prec:
return False

                if prev_prec == main_op_prec and prev_op in ('*', '/'):
return False

    post_op_match = None
    if end_index < len(clean_rhs):
        post_expr = clean_rhs[end_index:].lstrip()
post_op_match = re.match(r'([+\-*/%])', post_expr)

    if post_op_match:
        post_op = post_op_match.group(1)
post_prec = get_precedence(post_op)

        if post_prec > main_op_prec:
return False
```

```
    return True



def optimize_code(source_code: str) -> Tuple[str, List[Tuple[str, str, str, str]]]:
    optimized_code = source_code
substitutions = []      lines =
optimized_code.split('\n')

    calculated_expressions = {}

    final_optimized_lines = []

    for line in lines:
        indent_match = re.match(r'^(\s*)', line)        indentation =
indent_match.group(1) if indent_match else ""

        line_stripped = line.strip()

        if not line_stripped:
            final_optimized_lines.append(line)              continue        if line_stripped.startswith(('#Alp',
'@Vic', '~Adv', '//', '/*')) or line_stripped in ('{', '}', 'then', 'else'):
            final_optimized_lines.append(line)
continue

        if '=' not in line_stripped and not line_stripped.startswith(('if', 'while', 'do')):
            final_optimized_lines.append(line)
continue
```

```
    if '=' in line_stripped and not line_stripped.startswith(('if', 'while', 'do')):
try:
            lhs, rhs_full = line_stripped.split('=', 1)
lhs = lhs.strip()               rhs =
simplify_rhs(rhs_full)

            optimized_rhs = rhs

            sorted_expressions = sorted(calculated_expressions.items(), key=lambda item: len(item[0]),
reverse=True)

            for expr_source, var_source in sorted_expressions:

               if expr_source in optimized_rhs and expr_source != optimized_rhs and lhs != var_source:

                  if is_safe_to_substitute(optimized_rhs, expr_source):

                     optimized_rhs = optimized_rhs.replace(expr_source, var_source, 1)
substitutions.append((expr_source, lhs, var_source, "CSE_SUBSTITUTION"))

break



            if any(op in rhs for op in OP_PREC.keys()):
               calculated_expressions[rhs] = lhs

            final_optimized_lines.append(f"{indentation}{lhs} = {optimized_rhs};")

         except Exception:
```

```
            final_optimized_lines.append(line)

        elif line_stripped.startswith('if') or line_stripped.startswith('while'):

            match_cond = re.search(r'\(.*?\)', line_stripped)

            if match_cond:
                cond_expr_full = match_cond.group(0)
optimized_cond_expr = cond_expr_full

                sorted_expressions = sorted(calculated_expressions.items(), key=lambda item: len(item[0]),
reverse=True)

                for expr_source, var_source in sorted_expressions:
if expr_source in optimized_cond_expr:

                        if is_safe_to_substitute(optimized_cond_expr.strip('()'), expr_source):
optimized_cond_expr = optimized_cond_expr.replace(expr_source, var_source, 1)

                            substitutions.append((expr_source, line_stripped.split('(')[0], var_source,
"CSE_SUBSTITUTION"))

break

                optimized_line = line.replace(cond_expr_full, optimized_cond_expr, 1)
final_optimized_lines.append(f"{indentation}{optimized_line.strip()}")

            else:
                final_optimized_lines.append(line)
```

```
    else:
        final_optimized_lines.append(line)

optimized_code_final = '\n'.join(final_optimized_lines)

return optimized_code_final, substitutions
```

Código Ensamblador

```
import re
try:
    from .lexer import lex, Token
except ImportError:
    from lexer import lex, Token

JUMP_MNEMONICS = {
    "==": "EQ", "!=": "NE", ">": "GT", "<": "LT", ">=": "GE", "<=": "LE",
    "EQ": "EQ", "NE": "NE", "GT": "GT", "LT": "LT", "GE": "GE", "LE": "LE",
}

class Operand:
    def __init__(self, val, kind):
        self.val = val
        self.kind = kind

class AssemblyGenerator:
    def __init__(self, source_code):
        if not isinstance(source_code, str):
            source_code = str(source_code)

        self.tokens = lex(source_code)
        self.pos = 0
        self.output = []
        self.label_counter = 0

        if self.tokens:
            self.current_token = self.tokens[0]
        else:
            self.current_token = Token("EOF", "", -1)

    def advance(self):
        self.pos += 1
```

```
        if self.pos < len(self.tokens):
            self.current_token = self.tokens[self.pos]
        else:
            self.current_token = Token("EOF", "", -1)

    def match(self, kind):
        if self.current_token.kind == kind:
            val = self.current_token.lexeme
            self.advance()
            return val
        return None

    def new_label(self):
        self.label_counter += 1
        return f"RGN{self.label_counter}"

    def emit(self, instr, comment=""):
        line = f"{instr}"
        if comment:
            line = f"{line:<25} ; {comment}"
        self.output.append(line)

    def emit_label(self, label):
        self.output.append(f"{label}:")

    def parse_factor(self):
        tk = self.current_token
        if tk.kind == "LPAREN":
            self.advance()
            reg_op = self.parse_expression()
            self.match("RPAREN")
            return reg_op
        elif tk.kind in ("ID", "GEN", "INT", "REAL", "STRING"):
            val = tk.lexeme
            self.advance()
            return Operand(val, "LIT")
        else:
            self.advance()
            return Operand("AX", "REG")

    def parse_term(self):
        lhs = self.parse_factor()

        while self.current_token.kind in ("TIMES", "DIV", "MOD"):
            op = self.current_token.kind
```

```
            self.advance()

            if lhs.kind == "LIT":
                if op == "TIMES": self.emit(f"MOV AL, {lhs.val}")
                else:             self.emit(f"MOV AX, {lhs.val}")
                lhs.kind = "REG"
                lhs.val = "AX" if op != "TIMES" else "AL"

            if op == "TIMES" and lhs.val == "AX":
                self.emit("MOV AL, AL", "Prep Multiplicando")
            elif op in ("DIV", "MOD") and lhs.val == "AL":
                 self.emit("MOV AH, 0", "Extender AL a AX")
                 lhs.val = "AX"

            rhs = self.parse_factor()
            if rhs.kind == "LIT":
                self.emit(f"MOV BL, {rhs.val}")
            elif rhs.val != "BL":
                self.emit(f"MOV BL, {rhs.val}")
                pass

            if op == "TIMES":
                self.emit("MUL BL", "Resultado AX")
                lhs.val = "AX"
            elif op == "DIV":
                self.emit("DIV BL", "Cociente AL / Residuo AH")
                lhs.val = "AL"
            elif op == "MOD":
                self.emit("DIV BL", "Cociente AL / Residuo AH")
                lhs.val = "AH"

        return lhs

    def parse_expression(self):
        lhs = self.parse_term()

        while self.current_token.kind in ("PLUS", "MINUS"):
            op = self.current_token.kind
            self.advance()

            lhs_location = "LIT"
            if lhs.kind == "REG":
                if lhs.val == "AX":
                    self.emit("MOV BX, AX", "Guarda acumulado")
                    lhs_location = "BX"
```

```
            elif lhs.val in ("AL", "AH"):
                self.emit(f"MOV BL, {lhs.val}", "Guarda parcial 8b")
                self.emit("MOV BH, 0")
                lhs_location = "BX"

        rhs = self.parse_term()

        if op == "PLUS":
            if lhs_location == "BX":
                if rhs.kind == "LIT":
                    self.emit(f"ADD BX, {rhs.val}")
                elif rhs.val == "AX":
                    self.emit("ADD BX, AX")
                elif rhs.val in ("AL", "AH"):
                    self.emit("MOV AH, 0")
                    self.emit(f"ADD BX, AX")

                self.emit("MOV AX, BX")

            elif lhs_location == "LIT":

                if rhs.kind == "REG":
                    if rhs.val in ("AL", "AH"):
                        self.emit(f"MOV AX, {rhs.val}")
                        self.emit("MOV AH, 0")
                    self.emit(f"ADD AX, {lhs.val}")
                else:
                    self.emit(f"MOV AX, {lhs.val}")
                    self.emit(f"ADD AX, {rhs.val}")

        elif op == "MINUS":
            if lhs_location == "BX":
                if rhs.kind == "LIT":
                    self.emit(f"SUB BX, {rhs.val}")
                elif rhs.val == "AX":
                    self.emit("SUB BX, AX")
                elif rhs.val in ("AL", "AH"):
                    self.emit("MOV AH, 0")
                    self.emit("SUB BX, AX")
                self.emit("MOV AX, BX")

            elif lhs_location == "LIT":
                self.emit(f"MOV BX, {lhs.val}")

                if rhs.kind == "REG":
```

```
                    if rhs.val == "AX":
                        self.emit("SUB BX, AX")
                    elif rhs.val in ("AL", "AH"):
                        self.emit("MOV AH, 0")
                        self.emit("SUB BX, AX")
                else:
                    self.emit(f"SUB BX, {rhs.val}")

                self.emit("MOV AX, BX")

            lhs.kind = "REG"
            lhs.val = "AX"

        return lhs

    def parse_single_relational(self, l_true, l_false):
        lhs = self.parse_expression()

        if lhs.kind == "LIT":
            self.emit(f"MOV AX, {lhs.val}", "Cargar LHS")
        elif lhs.val != "AX":
             self.emit(f"MOV AX, {lhs.val}", "Mover LHS a AX")
        elif lhs.val == "AX":
             pass

        if self.current_token.kind in JUMP_MNEMONICS:
            op_kind = self.current_token.kind
            self.advance()

            rhs = self.parse_expression()

            cmp_operand = "BX"
            if rhs.kind == "LIT":
                cmp_operand = rhs.val
            else:
                if rhs.val == "AX":
                    self.emit("MOV BX, AX")
                else:
                    self.emit(f"MOV BX, {rhs.val}")

            self.emit(f"CMP AX, {cmp_operand}")

            mnemonic = JUMP_MNEMONICS.get(op_kind)
            self.emit(f"{mnemonic} {l_true}")
            self.emit(f"JMP {l_false}")
```

```
        else:
            self.emit("CMP AX, 0")
            self.emit(f"NE {l_true}")
            self.emit(f"JMP {l_false}")

    def parse_logic(self, l_then, l_else):
        l_next_check = self.new_label()

        has_and = False
        save_pos = self.pos
        for t in self.tokens[self.pos:]:
            if t.kind == "RPAREN": break
            if t.kind == "AND":
                has_and = True
                break

        if has_and:
            self.parse_single_relational(l_next_check, l_else)
            if self.current_token.kind == "AND":
                self.match("AND")
                self.emit_label(l_next_check)
                self.parse_single_relational(l_then, l_else)
        else:
            self.parse_single_relational(l_then, l_else)

    def parse_statement(self):
        tk = self.current_token

        if tk.kind in ("ID", "GEN"):
            var_name = tk.lexeme
            self.advance()
            if self.match("ASSIGN"):
                res = self.parse_expression()

                if res.kind == "LIT":
                    self.emit(f"MOV AX, {res.val}", "Cargar literal")
                    self.emit(f"MOV {var_name}, AX")
                elif res.val == "AL":
                    self.emit(f"MOV {var_name}, AL")
                elif res.val == "AH":
                    self.emit(f"MOV {var_name}, AH")
                else:
                    if res.val != "AX": self.emit(f"MOV AX, {res.val}")
                    self.emit(f"MOV {var_name}, AX")
```

```
            self.match("SEMI")
            return

        elif tk.kind == "IF":
            self.advance()
            self.match("LPAREN")

            l_then = self.new_label()
            l_else = self.new_label()
            l_end = self.new_label()

            self.parse_logic(l_then, l_else)

            self.match("RPAREN")
            self.match("THEN")

            self.match("LBRACE")
            self.emit_label(l_then)
            while self.current_token.kind not in ("RBRACE", "EOF"):
                self.parse_statement()
            self.match("RBRACE")

            self.emit(f"JMP {l_end}")

            self.emit_label(l_else)
            if self.match("ELSE"):
                self.match("LBRACE")
                while self.current_token.kind not in ("RBRACE", "EOF"):
                    self.parse_statement()
                self.match("RBRACE")

            self.emit_label(l_end)
            return

        else:
            self.advance()

    def generate(self):
        while self.current_token.kind != "EOF":
            self.parse_statement()
        return "\n".join(self.output)

def generate_assembly(source_code):
    try:
        if not source_code or not source_code.strip(): return "; Codigo vacio"
```

```
        gen = AssemblyGenerator(source_code)
        return gen.generate()
    except Exception as e:
        return f"; Error: {e}"
```